tion process. The PHP-enabled framework library also includes helper methods, which will help you to test the applications using a wide range of scenarios. If you inspect the test directory of your application, it will have new directories named Feature and Unit. While the feature tests are mainly oriented to test a large chunk of the code, the unit tests are aimed at a very small portion of your code with focus on a single method. If your application involves a full HTTP request to a JSON endpoint, you should adopt the feature tests.

Build data aware Laravel applications with PHPUnit

The installer includes a file named ExampleTest.php, which is included with both test directories. You should run phpunit on the command line to begin testing. The required environment variables are defined in the phpunit.xml file. Hence, Laravel will set the configuration environment automatically while running tests via the file. The framework also includes an ability to configure and cache the session during the testing phase. Hence, data from the session and cache won't persist while the application undergoes the testing process. Even though you can define custom environment variables based on the relevance of your application, the configuration cache should be cleared using the `config:clear` Artisan command. You can also create an env.testing file in the root, which will override the variables when running PHPUnit tests.

You can create a new Laravel test case with the help of the `make:test` Artisan command. To create a test in the feature directory, you need to use the following command:

```
php artisan make:test UserTest1
```

If you would like to create a test in the unit directory, you simply need to prefix `--unit` at the end of the above command. You can define test methods using `PHPUnit` after the generation of the test. The phpunit command should be executed from the terminal to run the tests. To create a basic test, you should call the `Tests\Unit` namespace and `assertTrue()` method.

Working with HTTP Testing

The Laravel framework ships with a fluent API to initiate HTTP requests to your application. You can define a test case as shown below:

```
public function testBasicTest()
```